OPNsense Firewall & Network Security Lab

Cybersecurity Home Lab | Oracle VirtualBox | OPNsense 26.7 | October 2026

Project status: COMPLETED — documented configuration and verification activities. Additional end-to-end WAN testing is identified separately.

1. Executive Summary

I designed and implemented a virtual network security lab to explore host-based firewall filtering, rule precedence, network segmentation, and perimeter firewall configuration. I first tested UFW on Xubuntu, using Kali Linux as a client to observe how default-deny policies and explicit allow/deny rules affect SSH and HTTP access. I then installed OPNsense, assigned its WAN and LAN interfaces using MAC address verification, connected Xubuntu to a dedicated internal segment, and configured an inbound HTTPS rule on the OPNsense WAN interface.

The sequence was intentional: host-level firewall exercises established how traffic is matched against rules; the OPNsense phase applied the same principles to a separate firewall appliance positioned between virtual network segments. Screenshots document the configuration and the tests actually performed.

2. Lab Environment and Addressing

System / interface	Address / MAC	Network / role
Kali Linux	172.30.1.21/24	Testing; SSH/HTTP test client
Xubuntu (before segmentation)	172.30.1.20/24	Testing; UFW-protected host
AlmaLinux	172.30.1.22/24	Testing; existing VM
OPNsense WAN / em1	172.30.1.22/24 (DHCP) | 08:00:27:20:49:39	Testing; Adapter 2
OPNsense LAN / em2	192.168.1.1/24 | 08:00:27:07:93:97	InsideFirewall; Adapter 3
OPNsense em0	08:00:27:ba:ab:d3	Adapter 1; VirtualBox NAT
Xubuntu enp0s9 (after change)	192.168.1.107/24	InsideFirewall; internal client

Addressing note: the initial environment inventory also lists AlmaLinux as 172.30.1.22, the same address later shown for OPNsense WAN. The screenshots do not establish whether both devices were simultaneously active on that address. This potential conflict should be checked before future multi-VM tests. The Xubuntu IP changed when its adapter was moved from Testing to InsideFirewall.

VirtualBox networks: NAT provides an optional upstream connection; Testing is the existing shared internal lab segment; InsideFirewall is the separate internal segment used by OPNsense LAN and Xubuntu. These labels refer to VirtualBox network names, not automatically to unrestricted Internet or production WAN access.

3. Preparation and Installation

3.1 Baseline connectivity and recovery point

Before introducing the firewall, I verified that Xubuntu could reach Kali Linux at 172.30.1.21 with five successful ICMP replies and 0% packet loss. I also created an Xubuntu snapshot named “Before Lab 4 - Firewall Configuration” so I could restore the VM if a network change caused problems.

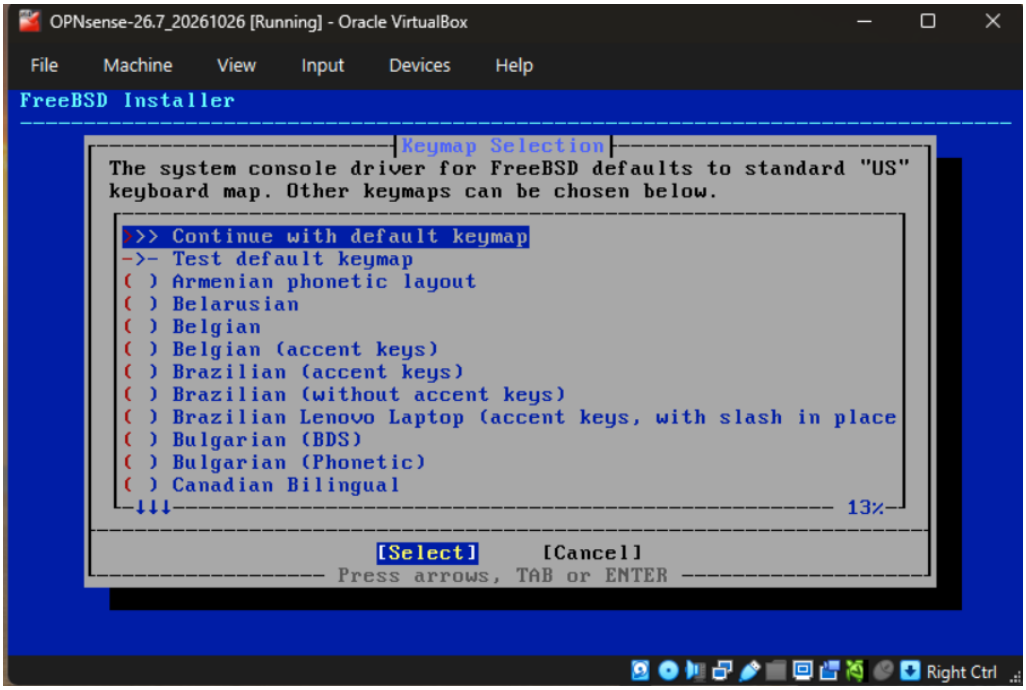
3.2 Installation image verification

I downloaded the OPNsense 26.7 amd64 DVD image from the official OPNsense website, checked its SHA-256 hash against the published checksum using PowerShell Get-FileHash, and extracted the .iso.bz2 archive with 7-Zip. This reduced the risk of using a corrupted installation image. The original document records the hash comparison as passed, but does not include the actual hash value or a verification screenshot.

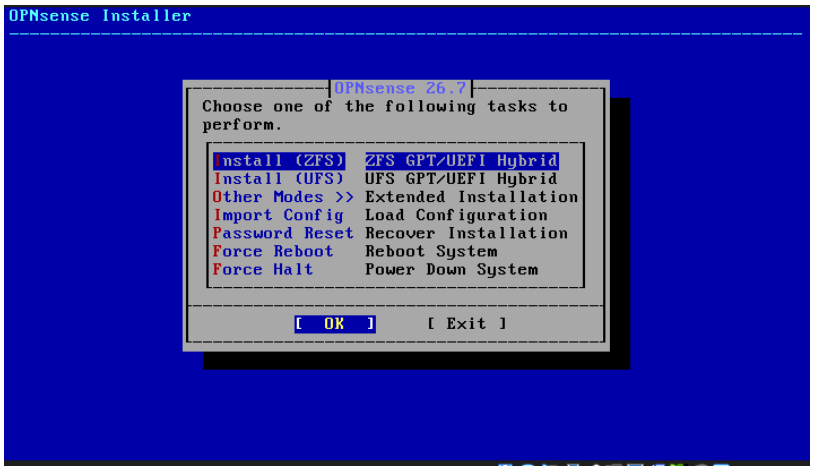
PowerShell command: Get-FileHash "C:\Users\longl\Downloads\OPNsense-26.7-dvd-amd64.iso.bz2" -Algorithm SHA256

3.3 Virtual machine deployment

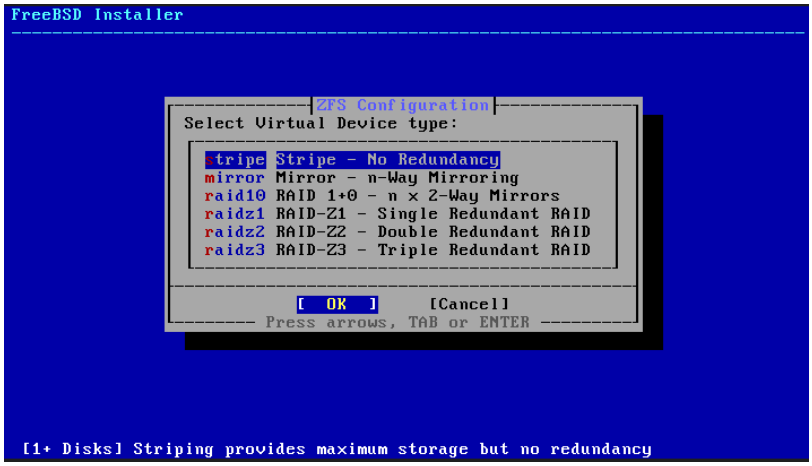
I created OPNsense-26.7_20261026 as a FreeBSD (64-bit) VM with 4 GB RAM, 2 vCPUs, and a dynamically allocated 32 GB virtual disk. I attached the verified installation ISO and installed OPNsense using ZFS on the single virtual disk. I configured the root account and completed the installer.



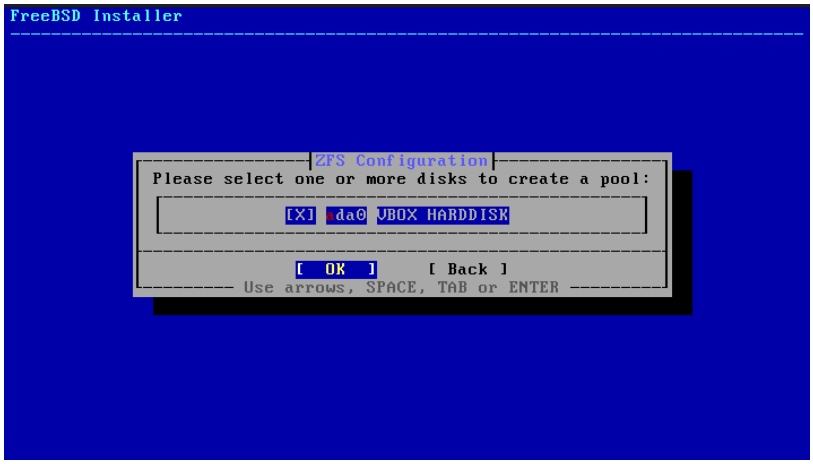
Installer: default keyboard layout



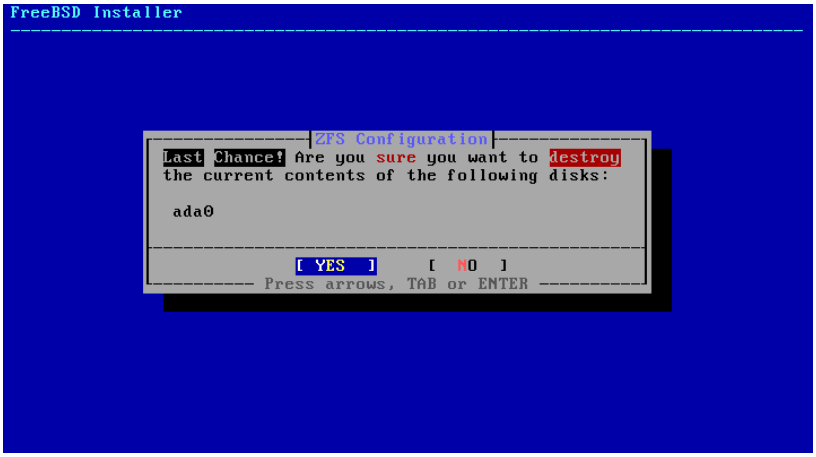
ZFS installation method



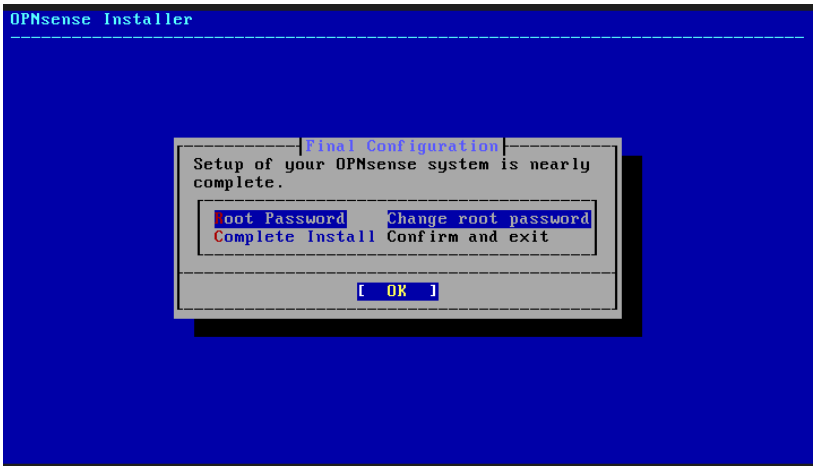
ZFS single-disk layout



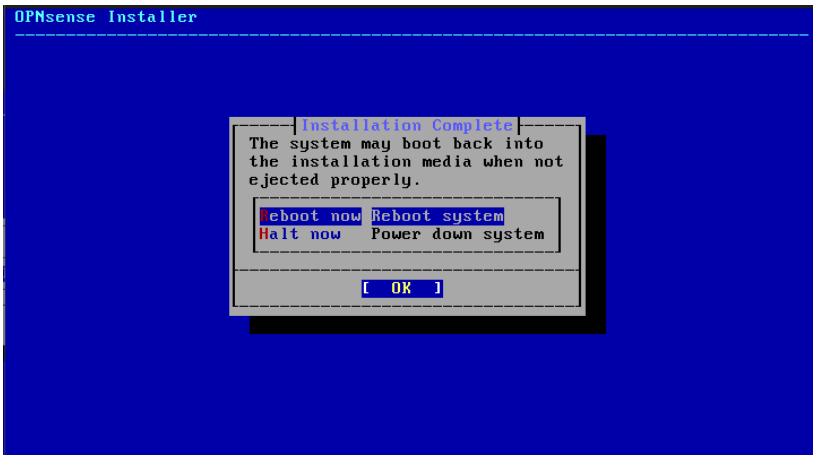
Virtual hard disk selection



Disk erase confirmation



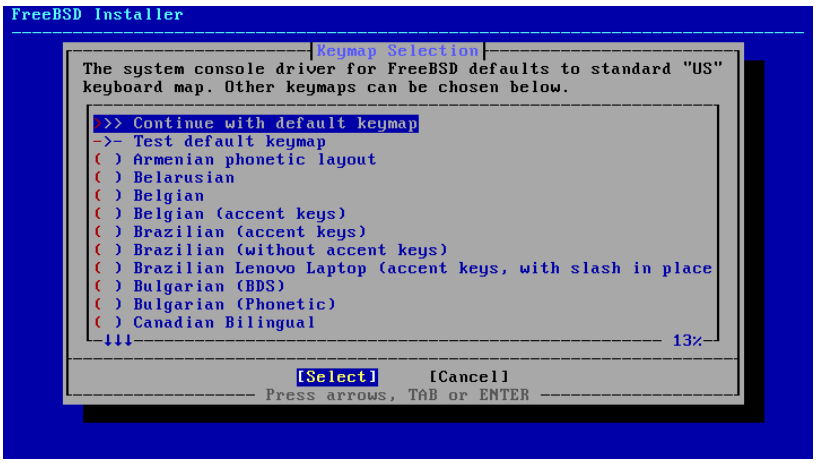
Root password configuration



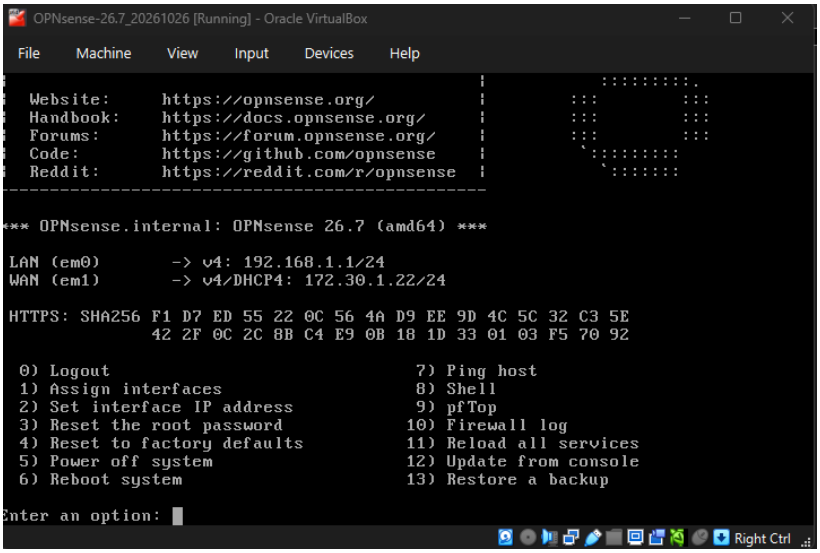
Installation completion

3.4 Installation troubleshooting

On the first reboot, the VM returned to the installer because the ISO remained mounted. I powered down the VM, removed the ISO from the virtual optical drive, and restarted it. OPNsense then booted from its installed virtual disk and displayed the console menu. This was a boot-media issue, not a failed installation.



Installer reappeared after reboot



OPNsense console after boot

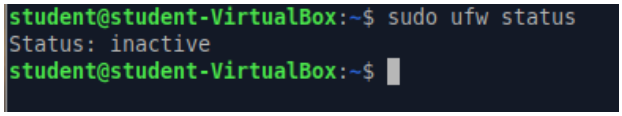
Installation verification: PASSED. The initial console showed LAN on em0 and WAN on em1; these were subsequently reassigned in Section 5. The final interface mapping is the authoritative configuration for this report.

4. Part A — Host-Based Firewall with UFW

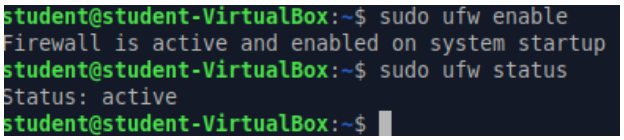
Objective: demonstrate default-deny behavior, service-specific exceptions, and rule precedence on Xubuntu before configuring the separate OPNsense firewall.

4.1 Default policy and baseline service testing

I checked UFW, enabled it, and inspected the active policy with sudo ufw status verbose. The firewall used deny (incoming) and allow (outgoing). I verified that the SSH service was running. From Kali Linux, ICMP echo requests to Xubuntu (172.30.1.20) succeeded, but SSH to TCP port 22 timed out. This demonstrated that successful ping replies do not imply that application ports are reachable.



Initial UFW status



UFW enabled

```
student@student-VirtualBox:~$ sudo ufw status
Status: active
student@student-VirtualBox:~$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enabled)
     Active: active (running) since Mon 2026-10-26 19:19:57 PDT; 5min ago
TriggeredBy: ● ssh.socket
       Docs: man:sshd(8)
             man:sshd_config(5)
    Process: 2646 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 2648 (sshd)
      Tasks: 1 (limit: 3453)
     Memory: 2.2M (peak: 5.0M)
        CPU: 33ms
     CGroup: /system.slice/ssh.service
             └─2648 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Oct 26 19:19:57 student-VirtualBox systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
Oct 26 19:19:57 student-VirtualBox sshd[2648]: Server listening on 0.0.0.0 port 22.
Oct 26 19:19:57 student-VirtualBox sshd[2648]: Server listening on :: port 22.
Oct 26 19:19:57 student-VirtualBox systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
Oct 26 19:24:53 student-VirtualBox sshd[2867]: Accepted publickey for student from 172.30.1.21 port 35658 >
Oct 26 19:24:53 student-VirtualBox sshd[2867]: pam_unix(sshd:session): session opened for user student(uid>
student@student-VirtualBox:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip
```

SSH service and default UFW policy

```
┌──(kali㉿kali)-[~]
└─$ ping -c 5 172.30.1.20
PING 172.30.1.20 (172.30.1.20) 56(84) bytes of data.
64 bytes from 172.30.1.20: icmp_seq=1 ttl=64 time=0.758 ms
64 bytes from 172.30.1.20: icmp_seq=2 ttl=64 time=0.930 ms
64 bytes from 172.30.1.20: icmp_seq=3 ttl=64 time=0.640 ms
64 bytes from 172.30.1.20: icmp_seq=4 ttl=64 time=0.817 ms
64 bytes from 172.30.1.20: icmp_seq=5 ttl=64 time=1.17 ms

── 172.30.1.20 ping statistics ──
5 packets transmitted, 5 received, 0% packet loss, time 4039ms
rtt min/avg/max/mdev = 0.640/0.862/1.168/0.179 ms

┌──(kali㉿kali)-[~]
└─$ ssh student@172.30.1.20
ssh: connect to host 172.30.1.20 port 22: Connection timed out
```

Ping succeeded while SSH was blocked

4.2 Permit SSH and HTTP

I added an OpenSSH allow rule and an HTTP allow rule, then verified the effective UFW policy. The commands were sudo ufw allow OpenSSH, sudo ufw allow 80/tcp, and sudo ufw status verbose. From Kali Linux, SSH authentication succeeded and curl -I http://172.30.1.20 returned HTTP/1.1 200 OK from Apache. The tests verified the two permitted services while the general inbound deny policy remained in place.

```
student@student-VirtualBox:~$ sudo ufw allow OpenSSH
[sudo] password for student:
Rule added
Rule added (v6)
student@student-VirtualBox:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

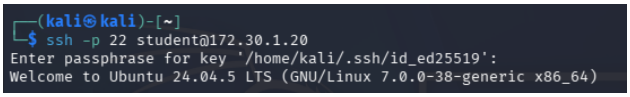
To                         Action      From
--                         ------      ----
22/tcp (OpenSSH)           ALLOW IN    Anywhere
22/tcp (OpenSSH (v6))      ALLOW IN    Anywhere (v6)
```

OpenSSH exception

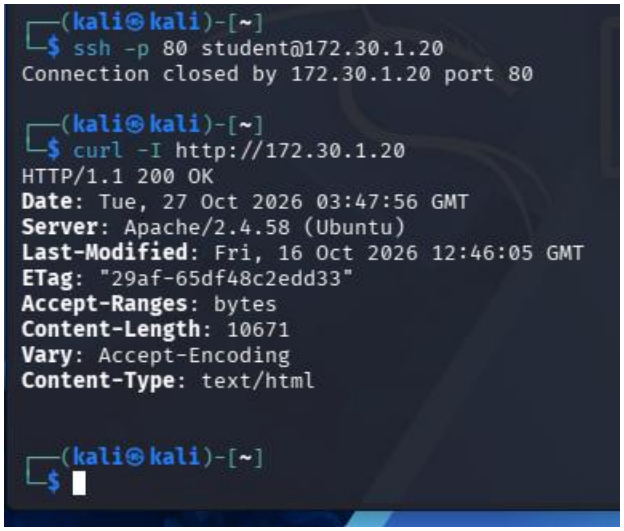
```
student@student-VirtualBox:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To                         Action      From
--                         ------      ----
22/tcp (OpenSSH)           ALLOW IN    Anywhere
80/tcp                     ALLOW IN    Anywhere
22/tcp (OpenSSH (v6))      ALLOW IN    Anywhere (v6)
80/tcp (v6)                ALLOW IN    Anywhere (v6)
```

HTTP exception



Successful SSH session



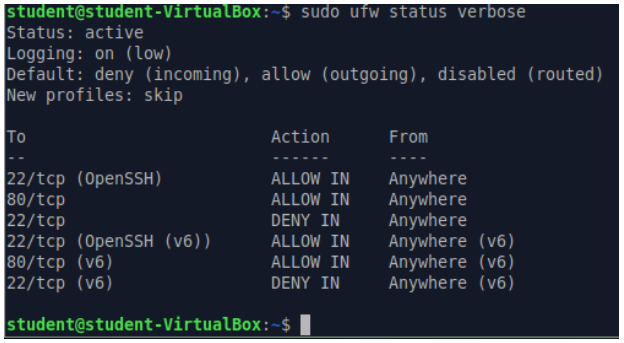
HTTP response from Apache

4.3 Rule ordering and final UFW policy

I added a deny rule for TCP port 22 below an existing allow rule. SSH remained accessible because the earlier matching allow rule took precedence. After inspecting numbered rules, I removed the earlier SSH allow entry and observed that a new SSH connection timed out. I then rebuilt the intended policy with default-deny inbound traffic and explicit SSH (22/tcp) and HTTP (80/tcp) allowances.



Added an SSH deny rule



Rule list shows conflicting SSH entries



SSH still accessible before rule removal

```
student@student-VirtualBox:~$ sudo ufw status numbered
Status: active

     To                         Action      From
     --                         ------      ----
[ 1] OpenSSH                    ALLOW IN    Anywhere
[ 2] 80/tcp                     ALLOW IN    Anywhere
[ 3] 22/tcp                     DENY IN     Anywhere
[ 4] OpenSSH (v6)               ALLOW IN    Anywhere (v6)
[ 5] 80/tcp (v6)                ALLOW IN    Anywhere (v6)
[ 6] 22/tcp (v6)                DENY IN     Anywhere (v6)
```

Numbered UFW rule order

```
student@student-VirtualBox:~$ sudo ufw delete 1
Deleting:
 allow OpenSSH
Proceed with operation (y|n)? y
Rule deleted
student@student-VirtualBox:~$
```

Deleted earlier allow rule

```
┌──(kali㉿kali)-[~]
└─$ ssh student@172.30.1.20
ssh: connect to host 172.30.1.20 port 22: Connection timed out
```

SSH connection timed out

```
student@student-VirtualBox:~$ sudo ufw allow ssh
Rule added
Rule added (v6)
student@student-VirtualBox:~$ sudo ufw allow http
Rule added
Rule added (v6)
student@student-VirtualBox:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To                         Action      From
--                         ------      ----
22/tcp                     ALLOW IN    Anywhere
80/tcp                     ALLOW IN    Anywhere
22/tcp (v6)                ALLOW IN    Anywhere (v6)
80/tcp (v6)                ALLOW IN    Anywhere (v6)
```

Final UFW configuration

Part A result: COMPLETED. The final status screenshot confirms UFW active with incoming SSH and HTTP permitted for IPv4 and IPv6. The experiment showed that rule order and default policy matter as much as the existence of individual rules.

5. Part B — OPNsense Network Segmentation

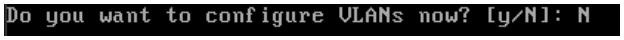
5.1 Identify and assign interfaces

I opened the OPNsense console and selected Assign interfaces. I declined LAGG and VLAN configuration because neither was required. Instead of assuming adapter numbers matched interface names, I compared the MAC addresses reported by OPNsense with the VirtualBox adapter settings. This identified em1 (08:00:27:20:49:39) as Testing/WAN and em2 (08:00:27:07:93:97) as InsideFirewall/LAN. The remaining em0 (08:00:27:ba:ab:d3) belonged to the NAT adapter and was not assigned as the active WAN or LAN in this step.

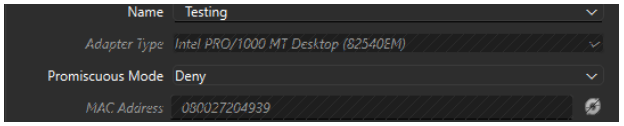
```
 0) Logout                              7) Ping host
 1) Assign interfaces                   8) Shell
 2) Set interface IP address            9) pfTop
 3) Reset the root password            10) Firewall log
 4) Reset to factory defaults          11) Reload all services
 5) Power off system                   12) Update from console
 6) Reboot system                      13) Restore a backup

Enter an option: 1
```

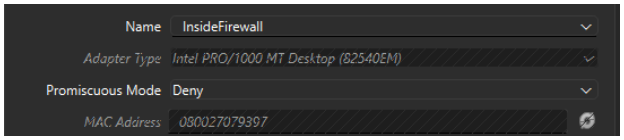
OPNsense console: interface assignment



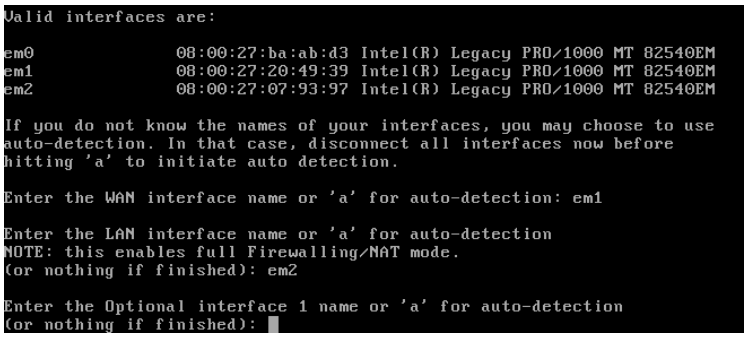
MAC-address comparison in VirtualBox



Selected em1 for WAN and em2 for LAN



Confirmed WAN/LAN mapping

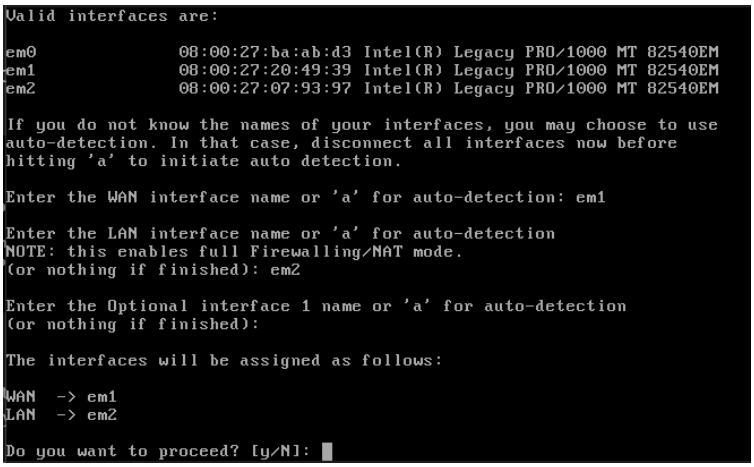


Console displays final WAN and LAN addresses

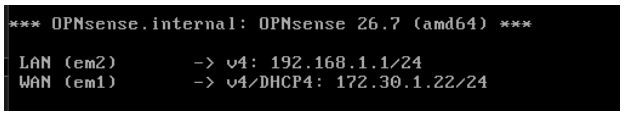
Final mapping: WAN em1 = 172.30.1.22/24 via DHCP on Testing; LAN em2 = 192.168.1.1/24 static on InsideFirewall. Interface assignment status: COMPLETED.

5.2 Move Xubuntu into the internal segment

With Xubuntu shut down, I changed its relevant VirtualBox adapter to Internal Network named InsideFirewall and confirmed that Virtual Cable Connected was enabled. After starting Xubuntu, ip -br addr showed enp0s9 at 192.168.1.107/24. This placed Xubuntu and the OPNsense LAN interface on the same 192.168.1.0/24 subnet.



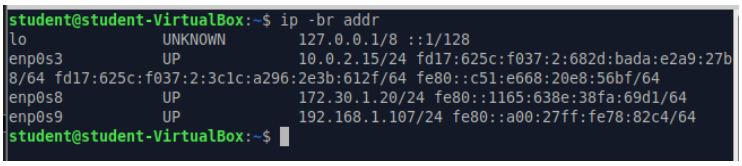
Xubuntu adapter on InsideFirewall



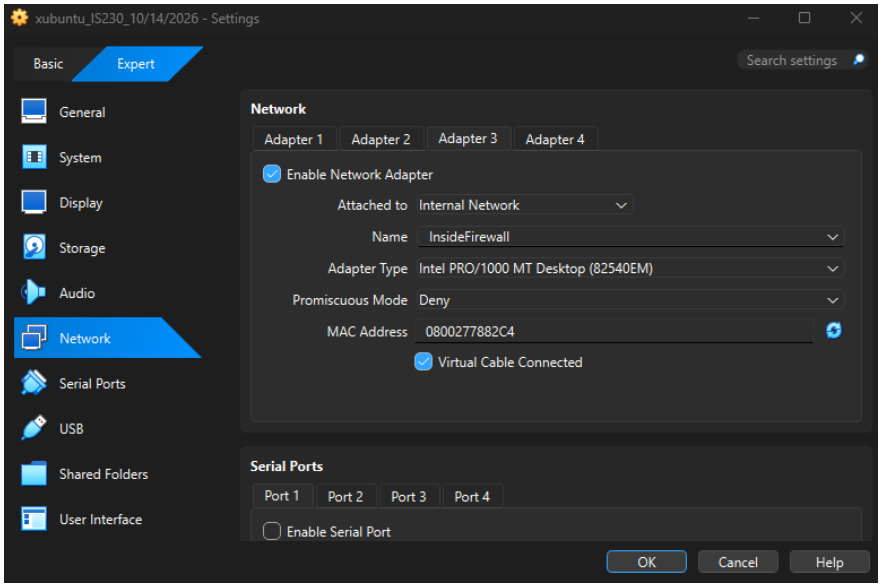
Xubuntu IP address on the internal subnet

5.3 Connectivity test and troubleshooting

My initial ping from Xubuntu to 192.168.1.1 failed (100% packet loss). I checked the addresses and confirmed UFW was inactive on Xubuntu during troubleshooting. After rebooting OPNsense, I repeated ping -c 3 192.168.1.1 and received all three replies with 0% loss. Rebooting restored connectivity, although the evidence does not conclusively identify the underlying fault.



Initial failed ping to OPNsense LAN



Successful ICMP connectivity after reboot

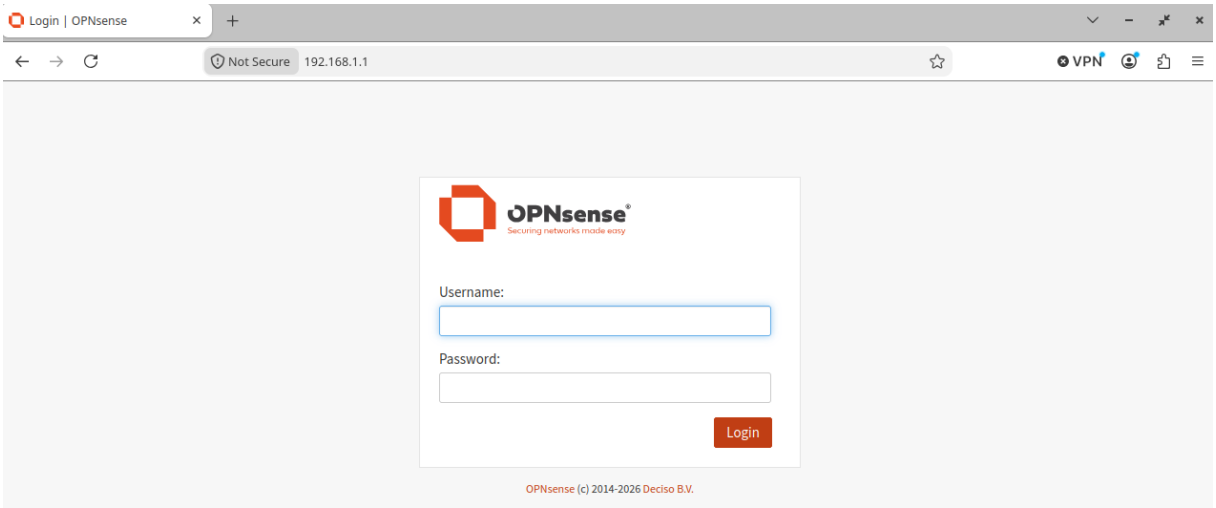
Part B result: COMPLETED. This validates direct Xubuntu-to-OPNsense LAN connectivity, not yet routed end-to-end traffic from InsideFirewall to Testing or the Internet.

6. Part C — OPNsense WAN Firewall Rule

Objective: create an inbound rule matching IPv4 TCP traffic with destination port 443 on the OPNsense WAN interface. This is a rule-configuration exercise; its scope and security implications should be distinguished from a full application connectivity test.

6.1 Web interface access

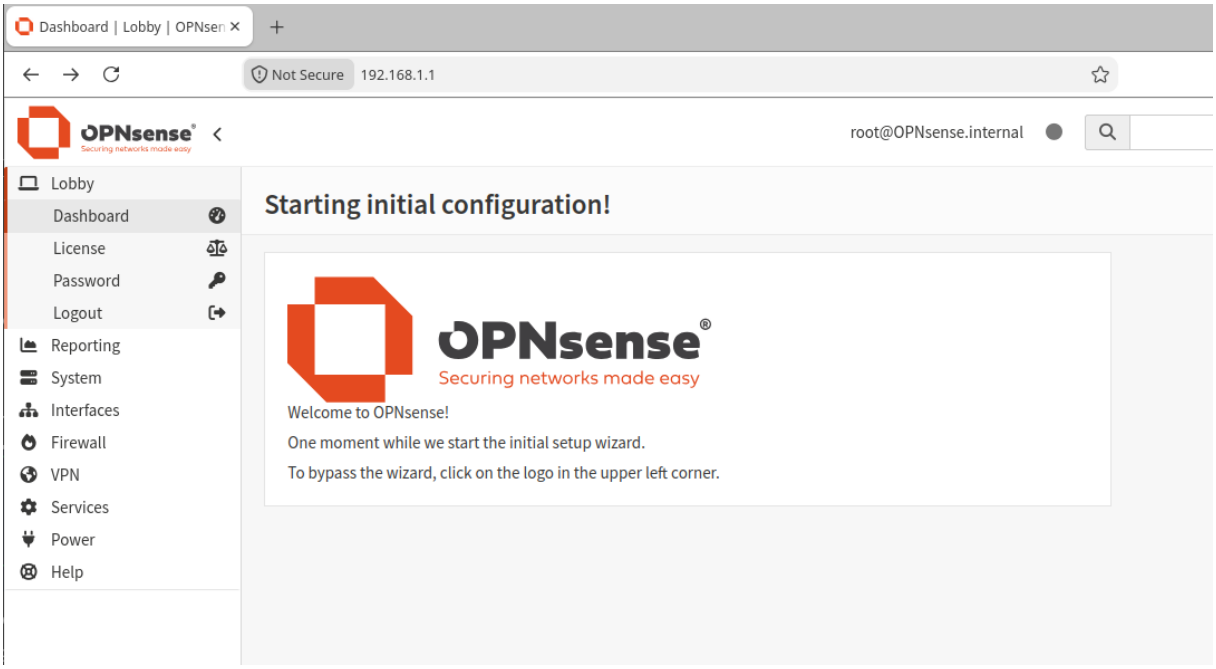
From Xubuntu, I opened the OPNsense management interface at 192.168.1.1 and authenticated using the root account and the password configured during installation. In a production deployment, HTTPS and a dedicated administrative account would be preferable to exposing administrative access broadly.



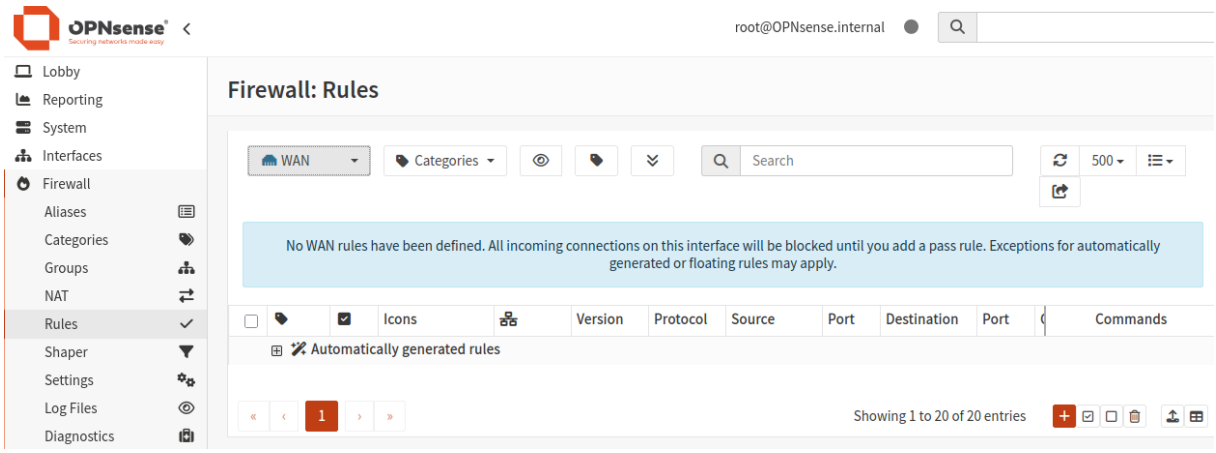
OPNsense Web UI login

6.2 Create the inbound HTTPS rule

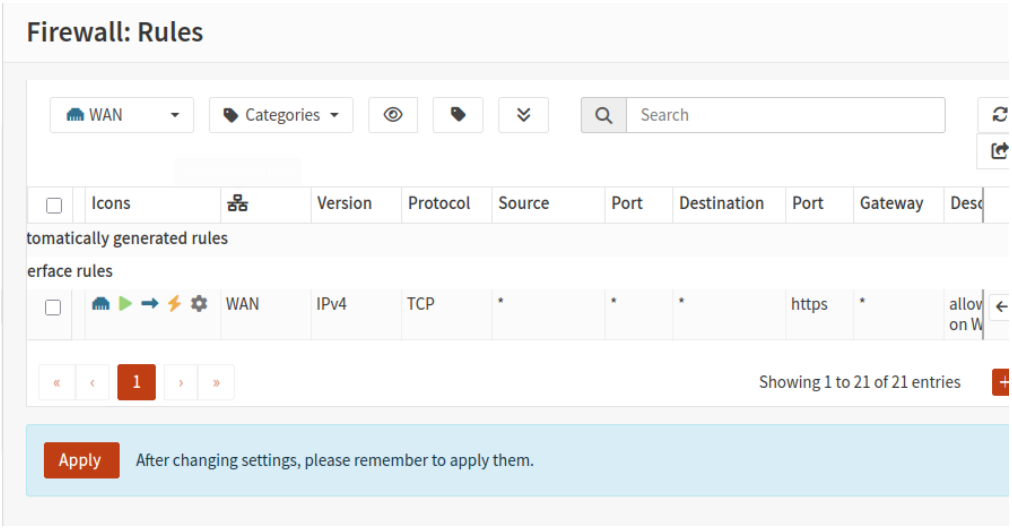
In Firewall > Rules, I selected WAN and added a rule with the following settings: Enabled; Action = Pass; Direction = In; IP version = IPv4; Protocol = TCP; Source = any; Source Port = any; Destination = any; Destination Port = HTTPS (443). I saved the rule and reviewed its entry in the WAN rule list.



OPNsense Web UI after login



WAN rule list before adding the rule



WAN rule list and Apply button

Important distinction: the screenshot shows the rule saved and an Apply button awaiting confirmation. The document does not independently show a successful external HTTPS connection or a post-Apply validation. Therefore, rule creation is verified, but deployment and traffic validation should not be claimed beyond the available evidence.

A TCP client typically initiates HTTPS with an ephemeral source port (for example 55000) and destination port 443. The reply reverses those ports. A stateful firewall tracks the permitted connection so return packets can be handled as part of that state. On an inbound WAN rule, however, destination port 443 is relevant only when traffic actually enters WAN; it does not by itself publish an internal web server. Depending on the desired architecture, a destination-specific rule, NAT/port forwarding, and service validation may be required.

Security note: a broad WAN Pass rule from any source to any destination on TCP 443 is appropriate only for a controlled lab exercise. A real deployment should restrict source and destination to the intended service and avoid unnecessarily broad exposure.

Part C status: RULE CREATED; APPLY AND END-TO-END VALIDATION NOT DOCUMENTED. This precise status reflects the screenshots rather than assuming a test occurred.

7. Results and Verification Matrix

Test / deliverable	Observed evidence	Outcome
Baseline Kali-to-Xubuntu ping	5/5 ICMP replies	PASS
UFW default deny / SSH blocked	SSH timeout with service running	PASS

SSH/HTTP exceptions	SSH login; HTTP 200 OK	PASS
UFW rule precedence	Allow precedence; blocked after removal	PASS
OPNsense installation	Booted to console after ISO removal	PASS
WAN/LAN interface mapping	MAC match and console address display	PASS
Xubuntu internal network	192.168.1.107/24 and ping 3/3	PASS
WAN TCP/443 rule creation	Rule visible in WAN table	PASS

8. Why the Lab Was Performed in This Order

Part A established the behavior of a host firewall: default-deny filtering, service-specific exceptions, and the importance of rule precedence. Part B then introduced a dedicated firewall with separate WAN and LAN interfaces, requiring careful MAC-to-interface mapping and subnet verification before any policy could be meaningfully tested. Part C built on both foundations by adding a protocol- and port-specific rule on the OPNsense WAN interface. This order helped isolate configuration problems: first establish a working service and understand filtering, then establish correct network connectivity, and only then add perimeter rules.

9. Lessons Learned and Conclusion

This lab strengthened my understanding of firewall configuration, network segmentation, and troubleshooting through hands-on experience. In Part A, I learned how UFW applies default-deny policies, permits specific services through firewall exceptions, and processes rules according to their precedence. I also observed that successful ICMP communication does not necessarily indicate that TCP services are accessible.

In Part B, I installed and configured OPNsense as a dedicated firewall, assigned WAN and LAN interfaces by matching their MAC addresses, and connected xUbuntu to the internal network. During troubleshooting, I resolved an installation boot issue caused by a mounted ISO and restored LAN connectivity by rebooting OPNsense after interface assignment.

Finally, I created and verified a WAN firewall rule permitting inbound HTTPS traffic on TCP port 443. All required lab exercises were completed and documented with configuration screenshots and observed results.

Overall, this project provided practical experience with Linux host-based firewalls, network interface configuration, virtual network segmentation, firewall rule management, and systematic troubleshooting. More importantly, it reinforced the importance of verifying network behavior through actual testing rather than relying solely on configuration settings.